

Filtering, Sorting & Grouping Data using Pandas

A Complete Guide to Data Manipulation with Real-Time Examples



FILTERING DATA



SORTING DATA



GROUPING DATA



**REAL-TIME
APPLICATIONS**

WHY FILTERING, SORTING & GROUPING MATTER

Real-World Data Challenges Require Powerful Tools



THE CHALLENGE

Imagine a school with **10,000 student records**. Finding specific data manually is impossible and highly inefficient.

- Find students scoring above 80
- Rank students by marks
- Calculate department-wise averages



THE SOLUTION

Pandas makes all these operations easy with **built-in, efficient functions**.

- These three operations are the foundation of Data Analysis
- Machine Learning
- Business Intelligence & Reporting

OUR SAMPLE DATASET

Setting Up a Student Records Dataset

Python Code

```
import pandas as pd

data = {
    "Name":
    ["Priya", "Rahul", "Sneha", "Kiran", "Asha"],
    "Department":
    ["AI", "DS", "AI", "Python", "DS"],
    "Age": [22, 21, 23, 20, 24],
    "Marks": [85, 92, 78, 88, 95]
}

df = pd.DataFrame(data)
# print(df) to see the output
```

DataFrame Output

Name	Department	Age	Marks
Priya	AI	22	85
Rahul	DS	21	92
Sneha	AI	23	78
Kiran	Python	20	88
Asha	DS	24	95

FILTERING DATA — SELECTING ROWS BY CONDITION

Filtering Means Keeping Only What You Need

WHAT IS FILTERING?

Filtering means selecting only the rows that satisfy a specific condition. Instead of viewing the entire dataset, we display only the required records.

REAL-LIFE EXAMPLES

- ✓ Students scoring above 80
- ✓ Employees with salary > ₹50,000
- ✓ Customers from a specific city
- ✓ Products costing less than ₹1000



PYTHON IMPLEMENTATION

```
high_marks = df[df["Marks"] > 80]
```

Explanation:

The condition `df["Marks"] > 80` is evaluated for each row. Pandas returns only the rows where the condition is

True

Result: Returns Priya (85), Rahul (92), Kiran (88), Asha (95).

FILTERING WITH MULTIPLE CONDITIONS

Combining Conditions for Precise Data Selection

Logical Operators

Use logical operators to combine multiple filtering conditions:

& **AND** (both conditions must be true)

| **OR** (at least one condition must be true)

~ **NOT** (negate the condition)

```
# Find AI department students with marks above 80
result = df[(df["Department"]=="AI") &
            (df["Marks"]>80)]
```

Output Result

Only **Priya** (AI, 85) satisfies both conditions.

Name	Department	Marks
Priya	AI	85

SORTING DATA — ARRANGING IN ORDER

Sorting Helps You Analyze Data at a Glance

WHAT IS SORTING?

Arranging data in a specific order (**Ascending** or **Descending**).

Real-life Examples: Highest salary first, lowest product price, alphabetical names, ranking students by marks.

Ascending (Default)

```
df.sort_values("Marks")
```

Output: 78 → 85 → 88 → 92 → 95

Descending

```
df.sort_values("Marks", ascending=False)
```

Output: 95 → 92 → 88 → 85 → 78

SORTING BY MULTIPLE COLUMNS

Multi-Column Sorting for Complex Data Organization

Python Code

```
# Sort by Department first, then by Marks
df.sort_values(
    ["Department", "Marks"],
    ascending=[True, False]
)
```

How It Works

This method is perfect for complex datasets where a single column sort isn't enough.

- ▶ First sorts departments **alphabetically** (e.g., AI → DS → Python).
- ▶ Within each department, students are sorted by marks from **highest to lowest**.

Use Case

This is especially useful for creating structured reports,

GROUPING DATA — COMBINING SIMILAR ROWS

Groupby Creates Categories and Enables Aggregation



WHAT IS GROUPING?

Combining rows that share the same value, then performing calculations on each group.

WHY GROUP?

- Find average salary per department
- Total sales per city
- Number of employees per branch

FUNCTION USED:



THINK OF IT AS CATEGORIES

Grouping organizes scattered data into meaningful categories:

AI → Priya, Sneha

DS → Rahul, Asha

Python → Kiran

🔧 Pandas handles the heavy lifting automatically!

AGGREGATE CALCULATIONS WITH GROUPBY

Compute Mean, Max, Min, and Count Per Group

Average Marks by Department

```
df.groupby("Department")["Marks"].mean()
```

Department	Average Marks
AI	81.5
DS	93.5
Python	88.0

Multiple Aggregations

```
df.groupby("Department")["Marks"].agg(  
    ["mean", "max", "min", "count"]  
)
```

Department	Mean	Max	Min	Count
AI	81.5	85	78	2
DS	93.5	95	92	2
Python	88.0	88	88	1

REAL-TIME INDUSTRY APPLICATIONS

Filtering, Sorting & Grouping Power Every Industry



SCHOOL MANAGEMENT

- Students scoring above 90
- Department-wise average marks



BANKING

- Customers with balance > ₹1 lakh
- Branch-wise total deposits



SALES & E- COMMERCE

- Region-wise sales performance



HEALTHCARE

- Patients grouped by disease type
- Average age analysis

KEY INTERVIEW QUESTIONS

Test Your Knowledge — Common Interview Questions

? Q1. What is filtering?

Selecting rows based on a specific condition.

? Q2. Which function is used for sorting?

```
sort_values()
```

? Q3. Which function groups data?

```
groupby()
```

? Q4. Difference between filtering and grouping?

CONCEPT	DESCRIPTION
Filtering	- Removes unwanted rows based on conditions. - Returns only the matching records.
Grouping	- Combines similar rows into categories. - Used for calculations like sum, mean, count, max, and min.

CLASSROOM ACTIVITY & LEARNING OUTCOME

Practice What You've Learned

HANDS-ON ACTIVITY

Using the sample student dataset, students should:

- 1 Display students with Marks > 85.
- 2 Display students from the DS department.
- 3 Sort the dataset by Age (ascending).
- 4 Sort the dataset by Marks (descending).
- 5 Find the average marks for each department using `groupby()`.
- 6 Find the maximum marks in each department.
- 7 Count the number of students in each department.



LEARNING OUTCOME

Students will understand how to filter, sort, and group data using Pandas — **essential skills** for data cleaning, exploratory data analysis (EDA), and real-world business reporting.